BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Bùi Quyền Kiệt
MSSV / Lớp	3124720036

Mục tiêu học tập

• Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).

• Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.

• Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.

• Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.

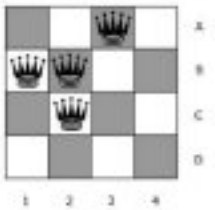
• Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Ban đầu: A3, B1, B2, C2. Hậu ở B2 đi chéo qua ô C3 trống đến D4.

Sau một vòng: A3, B1, C2, D4; số cặp hậu xung đột giảm từ 4 xuống 1.

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng

☒ B) Cực trị cục bộ (local optimum)

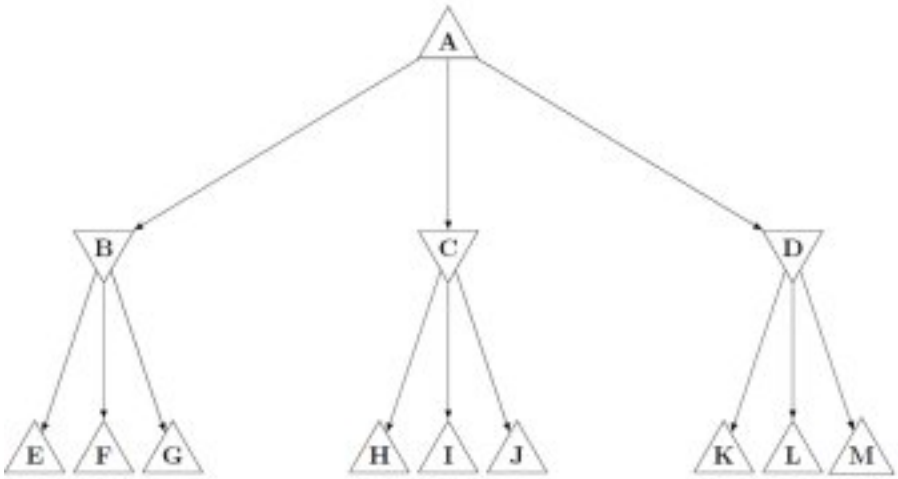
☐ C) Cực trị toàn cục (global optimum)

☐ D) Vai bằng (shoulder)

☐ E) Không thuộc các trường hợp trên

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D

☐ C → D → B

☒ D → C → B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Không thăm: I, J ở cây C; F, G ở cây B.

Sau khi duyệt D, α(A)=20; H=17 cắt I,J; E=5 cắt F,G.

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

☐ Có

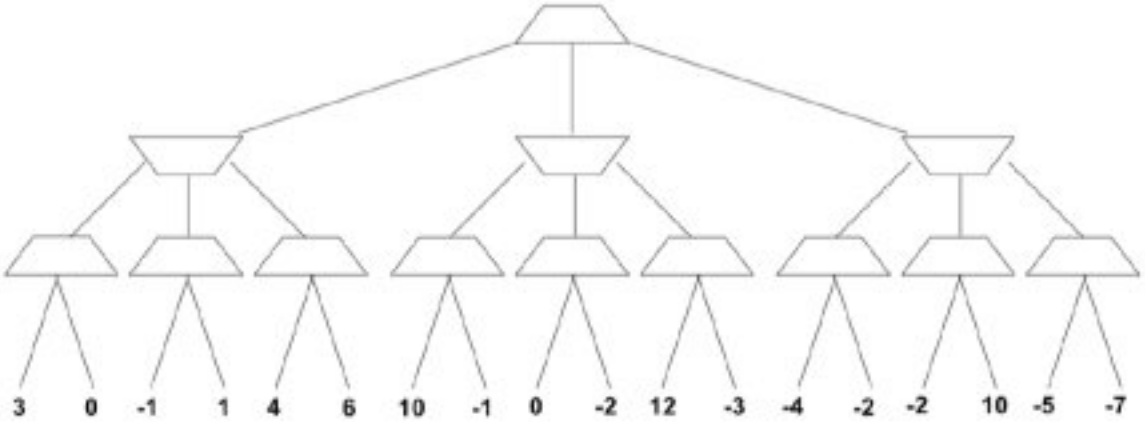
☒ Không

Không. Tìm kiếm có giới hạn độ sâu dùng giá trị đánh giá tại nút cắt, nên nước đi có thể khác Minimax trên toàn bộ cây; chỉ bảo đảm trùng khớp khi tìm đủ độ sâu (hoặc đánh giá chính xác).

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang

phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

• Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.

• Ghi giá trị được trả về tại mỗi nút trung gian.

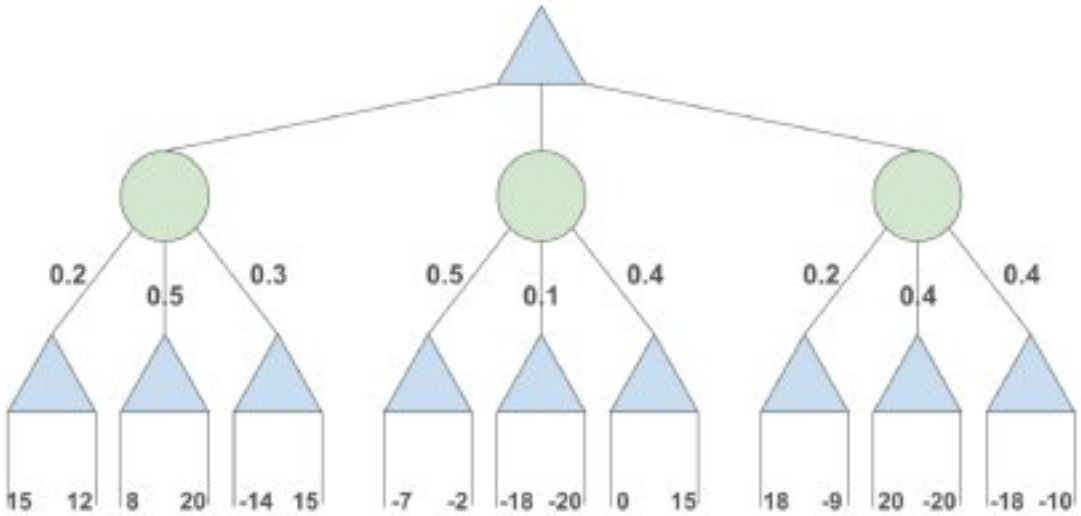
• Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.

• Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

A(MAX): ban đầu α=−∞, β=+∞. B(MIN): MAX(3,0)=3 (β=3), MAX(−1,1)=1 (β=1).
MAX(4,6) vào (α=−∞, β=1), gặp 4 thì α=4≥β nên cắt lá 6; B trả 1, A cập nhật α=1.
C(MIN), α=1: MAX(10,−1)=10 (β=10), MAX(0,−2)=0 (β=0≤α), cắt nhánh (12,−3).
D(MIN), α=1: MAX(−4,−2)=−2 (β=−2≤α), cắt (−2,10) và (−5,−7). A trả 1; chọn nhánh trái.

Bài 4. Expectimax và Chance Nodes

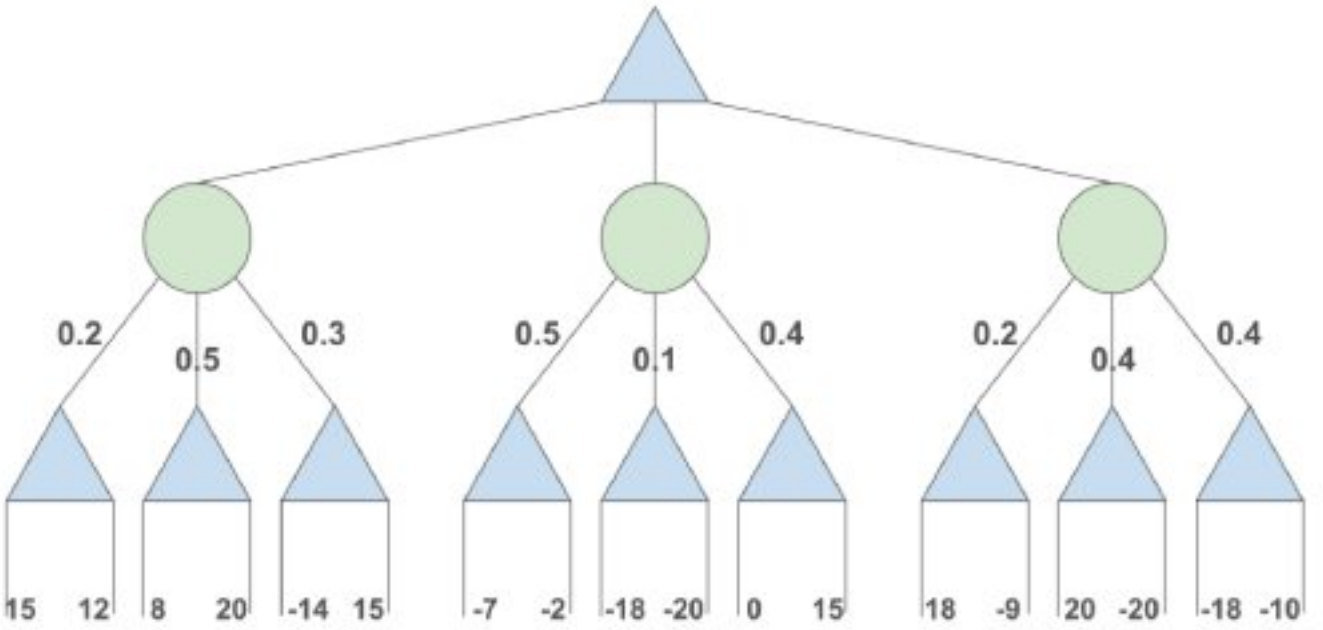
Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Các nút MAX bên trái: 15, 20, 15 ⇒ Chance trái = 0,2×15 + 0,5×20 + 0,3×15 = 17,5.
Ở giữa: MAX(−7,−2)=−2; MAX(−18,−20)=−18; MAX(0,15)=15.
Chance giữa = 0,5×(−2) + 0,1×(−18) + 0,4×15 = 3,2.
Bên phải: MAX=18,20,−10 ⇒ Chance phải = 0,2×18 + 0,4×20 + 0,4×(−10)=7,6; gốc MAX=17,5.
b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.
Không thể cắt tỉa nhánh nào mà vẫn bảo đảm kết quả chính xác khi chưa biết giới hạn giá trị lá.
Ở chance node, mọi nhánh có xác suất dương đều ảnh hưởng đến kỳ vọng; một giá trị lá chưa xét có thể thay đổi kết quả. Các nút MAX cũng cần xét đủ lá nếu không có cận trên/dưới hữu ích.
c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Chance trái = 17,5 là giá trị tốt nhất hiện có. Ở chance giữa, MAX trái = −2; cận trên là 0,5×(−2)+0,1×20+0,4×20=9 < 17,5, nên cắt hai nhánh còn lại: (−18,−20) và (0,15).
Chance phải không cắt được vì cận trên vẫn có thể đạt 19,6 > 17,5; tính đủ được 7,6. Gốc trả 17,5.

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.
Forward Checking chạy sau mỗi phép gán và chỉ loại giá trị xung đột ở các biến kề trực tiếp; không tiếp tục lan truyền qua các miền vừa bị thu hẹp. Vì vậy thường có chi phí thấp hơn.
AC-3 kiểm tra các cung (Xi,Xj), sửa miền và tiếp tục đưa các cung liên quan vào hàng đợi đến khi nhất quán cung. AC-3 lan truyền rộng hơn, phát hiện mâu thuẫn sớm hơn nhưng tốn công hơn.
b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV)

MRV chọn biến có ít giá trị hợp lệ còn lại nhất, tức biến bị ràng buộc mạnh nhất, để xử lý trước.
Cách này giúp phát hiện nhánh vô nghiệm sớm và thường giảm số phép thử/backtracking.
ii) Least Constraining Value (LCV)
LCV chọn giá trị loại bỏ ít lựa chọn nhất khỏi miền của các biến lân cận, giữ lại nhiều phương án cho các biến chưa gán và thường giúp tìm nghiệm với ít backtracking hơn.

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

• Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.

• Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.

• Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.

• Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C. • Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng. a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

Biến: $X_i=(t_i,r_i)$, $i\in\{A,B,C,D,E\}$; miền $D_i=\{1,2,3,4\}\times\{\text{quốc tế,nội địa}\}$.
Unary: t_B=1; t_D∈{3,4}; t_A∈{1,2}.
Binary: t_D<t_C; với mọi i≠j, không được đồng thời có $t_i=t_j$ và $r_i=r_j$
(hay $(t_i,r_i)\neq(t_j,r_j)$), nghĩa là không trùng giờ trên cùng đường băng.
Bổ sung hai ràng buộc sau:

• A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.

• D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.
Miền thời gian sau khi cố định đường băng: A={1,2}; B={1}; C={1,2,3,4};
D={3,4}; E={1,2,3,4}.
Đồ thị ràng buộc gồm các cạnh AB, AC, BC (cùng đường băng quốc tế), CD (t_D<t_C),
DE (cùng đường băng nội địa): tam giác A–B–C nối với chuỗi C–D–E.
c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.
Sau unary: A={1,2}, B={1}, C={1,2,3,4}, D={3,4}, E={1,2,3,4}.
AB loại 1 khỏi A ⇒ A={2}; AC và BC loại 2,1 khỏi C ⇒ C={3,4}. Từ D<C loại D=4,
nên D={3}; khi D=3 thì C phải lớn hơn 3 ⇒ C={4}. DE loại 3 khỏi E ⇒ E={1,2,4}.
Kết quả arc consistency: A={2}, B={1}, C={4}, D={3}, E={1,2,4}.

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo

MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

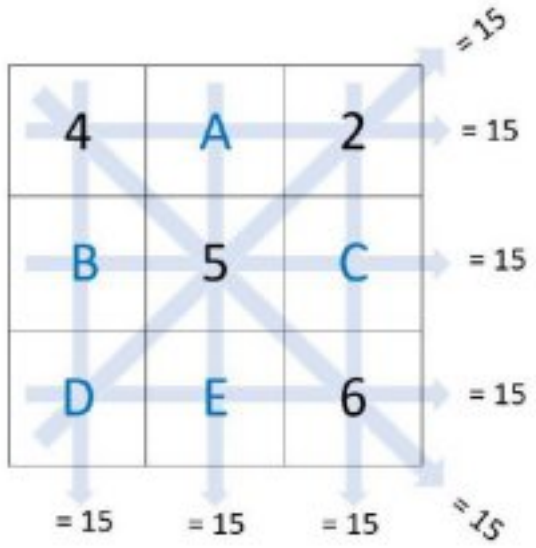
Thứ tự gán: (B,1) → (A,2) → (C,3) [dead end, hoàn tác] → (C,4) → (D,3) → (E,1).

Sau B=1, A còn {2}; sau A=2, C còn {3,4}. Chọn C=3 làm miền D rỗng vì cần D<C.

Hoàn tác C=3, thử C=4 ⇒ D={3}; sau D=3 thì E={1,2,4}, chọn E=1.

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n^2}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

Biến: A,B,C,D,E; miền ban đầu {1,3,7,8,9} (không dùng lại 2,4,5,6), kèm alldiff.

Ràng buộc tổng: A=9; B+C=10; D+E=9; B+D=11; A+E=10; C=7.

Đường chéo: 4+5+6=15 đã thỏa; 2+5+D=15 ⇒ D=8. Mỗi hàng/cột/đường chéo có tổng 15.

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.

Đồ thị nhị phân (bỏ alldiff): A—E (cột 2), E—D (hàng 3), D—B (cột 1), B—C (hàng 2).

Các ràng buộc đơn: A=9, C=7, D=8; cấu trúc đồ thị là đường đi A–E–D–B–C.

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

AC-3 thu hẹp A={9}, C={7}, D={8}; từ B+C=10 và B+D=11 suy ra B={3}; từ D+E=9

(đồng thời A+E=10) suy ra E={1}. Nghiệm Magic Square là [4,9,2; 3,5,7; 8,1,6].

Mỗi hàng, mỗi cột và hai đường chéo đều có tổng bằng 15.

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

• AJ = {Đỏ, Vàng, Xanh lá}

• Sunny = {Đỏ, Vàng, Hồng}

• Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

Backtrack = 0. MRV chọn Ellyse ({Đỏ,Hồng}); LCV chọn Hồng vì chỉ loại Hồng khỏi miền Sunny.

Sau đó Sunny chọn Đỏ (hòa với Vàng, ưu tiên theo thứ tự cầu vồng); AJ chọn Vàng trước Xanh lá.

Thứ tự gán: (Ellyse,Hồng) → (Sunny,Đỏ) → (AJ,Vàng).

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

Backtrack = 1. Maximum Remaining Values hòa AJ/Sunny, chọn AJ trước; Most Constraining Value chọn Đỏ.

Sunny còn {Vàng,Hồng}; chọn Hồng làm Ellyse hết màu nên gặp dead end và phải backtrack một lần.

Thử Sunny=Vàng rồi Ellyse=Hồng thì thành công. So với (a), tăng 1 lần do chọn giá trị ràng buộc mạnh nhất.

--- Hết ---